

SBA301

HƯỚNG DẪN HỌC TẬP DÀNH CHO SINH VIÊN

BUỔI 02 / SLOT 02

ReactJS Fundamentals - Vite - JSX - First React Application

Mục tiêu của tài liệu

Tài liệu này hướng dẫn sinh viên biết phải chuẩn bị gì trước lớp, thao tác từng bước trong buổi học, phối hợp với bạn học, thu thập evidence, sử dụng AI đúng cách, tự kiểm tra mức độ hiểu và chuyển kết quả sang Group Project.

Thông tin	Nội dung
Môn học	SBA301 - Integrate Single Page Application with Spring Boot
Baseline	SBA301-40 v2
Session	02 / 40 (Teaching Session 02 / 38)
Thời lượng	135 phút
Nội dung chính	Chapter 01 - Introduction to ReactJS + phần khởi đầu của Lab 01
Đầu ra tối thiểu	React/Vite app chạy; tạo component đầu tiên; JSX expression hoạt động; không có lỗi console liên quan; có Git commit; có evidence
Không học sâu hôm nay	Props, State, Hooks, Context, Router, Axios, Spring Boot backend

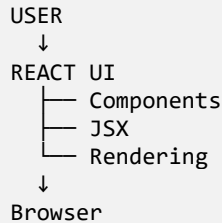
Nguyên tắc xuyên suốt: THINK FIRST → BUILD SMALL → VERIFY → EXPLAIN → COMMIT

1. Bạn sẽ học gì trong Buổi 02?

Big Picture

Buổi 01: nhìn toàn bộ hệ thống full-stack.

Buổi 02: zoom vào lớp FRONTEND và tự tạo ứng dụng React đầu tiên.



Sau buổi học, bạn phải làm được	Evidence
Giải thích React là gì và React khác Vite như thế nào	Trả lời Human Verification Gate
Tạo React project bằng Vite	Thư mục project + terminal output
Chạy development server	URL local mở được trên browser
Nhận biết index.html, main.jsx, App.jsx	Giải thích flow main.jsx → App → component
Viết JSX và những JavaScript expression	UI hiển thị dữ liệu từ biến/object
Tạo function component, import/export	CourseHeader và StudentProfile hoạt động
Debug lỗi cơ bản	Terminal/Console evidence + mô tả lỗi
Tạo Git checkpoint	git log -1 có commit phù hợp

2. Quy tắc học trong buổi này

- Không copy toàn bộ code rồi chạy. Mỗi thay đổi chỉ 1 bước nhỏ, sau đó kiểm tra browser/console.
- AI-0: phải tự dự đoán, tự vẽ hoặc tự code trước khi hỏi AI.
- Nếu có lỗi, thu thập evidence trước: command, terminal output, browser console, file + line.
- Không thêm Props/State/Hooks/Router vì chưa thuộc scope của Session 02.
- Mỗi nhóm/pair luân phiên Driver - Navigator - Reviewer để mọi người đều thao tác và giải thích.

3. Chuẩn bị trước lớp (10-20 phút trước buổi học)

Điều kiện đầu vào

Môi trường cơ bản từ Buổi 01 đã được cài/kiểm tra. Nếu còn FAIL, bạn vẫn tham gia phần mental model và JSX; đồng thời ghi rõ lỗi để xử lý theo checklist.

- ☐ Node.js chạy được: node -v
- ☐ npm chạy được: npm -v
- ☐ Git chạy được: git --version
- ☐ IDE/VS Code mở được thư mục SBA301
- ☐ Thư mục SBA301/labs tồn tại

□ Browser mở được và biết dùng F12 → Console

```
node -v
npm -v
git --version
```

4. Recall từ Buổi 01 - không dùng AI

Câu hỏi	Tự trả lời
Frontend là gì?	_____
React nằm ở layer nào?	_____
Java và JavaScript có giống nhau không?	_____
Node.js/npm có vai trò gì trong môi trường React?	_____
React sau này giao tiếp với Spring Boot bằng gì?	_____
Browser khác server ở điểm nào?	_____

5. Mental model: React là gì?

Định nghĩa tối thiểu cần nhớ

React là JavaScript library dùng để xây dựng user interface. React không phải programming language, database, backend server hay CSS framework.

```
DATA
↓
COMPONENT
↓
UI
```

5.1. Tại sao cần React?

Một trang tĩnh có thể viết bằng HTML. Một web application lớn cần tổ chức UI thành các phần nhỏ, tái sử dụng và cập nhật theo dữ liệu. React hỗ trợ tư duy component-based và declarative UI.

```
PAGE
├── Header
├── StudentCard
├── CourseList
└── Footer
```

5.2. Declarative UI ở mức người mới

Imperative thinking	React/declarative thinking
Tìm element rồi sửa text	Mô tả UI cần trông như thế nào với dữ liệu hiện tại
Tự add/hide/show từng phần	Component trả về mô tả UI bằng JSX
Tập trung thao tác DOM	Tập trung dữ liệu → component → UI

5.3. React application flow hôm nay

```

Browser
↓
index.html
↓
main.jsx
↓
React root
↓
App
↓
Child Components
↓
Rendered UI

```

Bạn cần giải thích được

main.jsx là nơi khởi tạo/render React app; App.jsx là top-level component trong starter project; các component con tạo thành component tree.

6. Vite, Node và npm: hiểu trước khi gõ lệnh

Công cụ	Vai trò trong buổi 02
React	UI library: component + rendering
Vite	Scaffold project, development/build tooling, dev server
Node.js	Runtime/tooling environment dùng bởi Vite/npm trong development
npm	Cài package và chạy scripts
Browser	Hiển thị UI và cung cấp Console/Elements để kiểm tra

React ≠ Vite

React là library để xây UI. Vite là công cụ giúp tạo/chạy/build frontend project. Vite project có thể dùng React nhưng Vite không phải React.

7. Thực hành A - Tạo React/Vite project từ số 0

Mục tiêu

Kết thúc phần này: bạn có project demo-01, dependencies đã cài, npm run dev chạy và browser hiển thị baseline page.

Bước A1 - Mở đúng workspace

1. Mở IDE/VS Code.
2. Open Folder: SBA301/labs.
3. Mở Terminal trong IDE.
4. Kiểm tra terminal đang ở đúng thư mục.

```
Get-Location      # PowerShell
# hoặc pwd trên terminal hỗ trợ Unix-like
```

PASS khi

Đường dẫn terminal kết thúc bằng SBA301/labs (hoặc thư mục labs tương đương của bạn).

Bước A2 - Kiểm tra Node/npm

```
node -v
npm -v
```

PASS khi

Cả hai lệnh trả về version. Nếu command not found/not recognized: dừng bước scaffold và sửa environment trước.

Bước A3 - Tạo starter project

Lệnh trong tài liệu Lab 01/baseline:

```
npm create vite@latest demo-01
```

Lưu ý tương thích môi trường

Nếu máy đang dùng version Node được khóa theo hướng dẫn môi trường của lớp và `latest` báo không tương thích, KHÔNG tự nâng/cài ngẫu nhiên. Dùng Vite version được giảng viên phê duyệt cho lớp hoặc lệnh đã quy định trong tài liệu cài đặt.

5. Chọn Framework: React.
6. Chọn Variant: JavaScript (trừ khi giảng viên quy định khác).
7. Không chọn TypeScript cho bài này nếu course baseline đang dùng JavaScript.

PASS khi

Thư mục demo-01 được tạo và có package.json, src/, index.html.

Bước A4 - Đi vào project và cài dependencies

```
cd demo-01
npm install
```

Bạn vừa làm gì?

npm đọc package.json và cài dependencies vào node_modules. Không sửa trực tiếp node_modules.

PASS khi

npm install hoàn tất không có error chặn; package-lock.json và node_modules xuất hiện (tùy môi trường).

Bước A5 - Chạy development server

```
npm run dev
```

Mental model

npm run dev → Vite Dev Server → local URL → Browser. Đây KHÔNG phải Spring Boot server.

8. Đọc local URL từ terminal.
9. Mở URL đó trên browser.
10. Nhấn F12 → Console.
11. Ghi evidence baseline.

Evidence	Ghi nhận
Dev URL	_____
Trang hiển thị	YES / NO
Terminal error chặn	YES / NO
Browser console relevant error	YES / NO

QUALITY GATE A

Chỉ chuyển sang sửa App.jsx khi baseline page đã mở được. Nếu baseline chưa chạy, đừng thay code để “đoán”.

8. Đọc cấu trúc project trước khi code

```
demo-01/
├── src/
│   ├── App.jsx
│   ├── main.jsx
│   └── ...
├── public/
├── index.html
├── package.json
└── vite.config...
```

Cấu trúc thật có thể hơi khác theo version Vite. Hãy đọc project thật của bạn, không cố ép giống một screenshot cũ.

File/Folder	Vai trò cần nhớ
index.html	Browser entry document; có root mounting point
src/main.jsx	Khởi tạo React root và render App
src/App.jsx	Top-level component của starter project
src/components/	Thư mục bạn sẽ tạo cho các component nhỏ
package.json	Scripts + dependencies + project/package metadata
node_modules/	Dependencies đã cài; không chỉnh tay
public/	Static assets theo nhu cầu

9. JSX - đọc, sửa và kiểm chứng

Định nghĩa

JSX là syntax extension thường dùng với React để mô tả UI bằng cú pháp HTML-like bên trong JavaScript. JSX không phải HTML thuần.

```
function App() {
  return (
    <h1>Hello SBA301</h1>
  );
}
```

9.1. JavaScript expression trong JSX

```
const courseName = "SBA301";

return (
  <h1>{courseName}</h1>
);
```

Quy tắc thực hành

{ ... } dùng để nhúng JavaScript expression vào JSX, ví dụ {name}, {1 + 2}, {student.name}, {isReady ? "Ready" : "Not Ready"}.

9.2. JSX khác HTML ở điểm cần biết hôm nay

HTML	JSX
class	className
Markup document	Syntax nằm trong JavaScript/React code
Không nhúng trực tiếp JS expression theo kiểu React	Dùng {expression}
Một số attribute theo HTML naming	Một số property/attribute dùng React/DOM naming

9.3. Fragment và cấu trúc return

```
return (
  <>
    <h1>SBA301</h1>
    <p>React Fundamentals</p>
  </>
);
```

PASS khi

Code compile; UI xuất hiện; không có lỗi JSX liên quan trong terminal/console.

10. Function Component - component đầu tiên của bạn

```
function Welcome() {
  return (
    <h2>Welcome to SBA301</h2>
  );
}
```

Mental model

Trong phạm vi hôm nay: component $\approx$ JavaScript function trả về mô tả UI bằng JSX.

10.1. Tên component phải bắt đầu bằng chữ hoa

```
<h1>Hello</h1>      // built-in DOM element
<Welcome />         // user-defined React component
```

10.2. Export $\rightarrow$ Import $\rightarrow$ Use

```
// Welcome.jsx
function Welcome() {
  return <h2>Welcome to SBA301</h2>;
}

export default Welcome;
```

```
// App.jsx
import Welcome from "./components/Welcome";

function App() {
  return <Welcome />;
}

export default App;
```

Bạn cần giải thích được

File component export một value/component; file khác import value đó; JSX dùng <Welcome /> để compose UI.

11. Thực hành B - My SBA301 Profile Application

Yêu cầu sản phẩm

Tạo UI gồm CourseHeader và StudentProfile. Hiển thị course code, topic, student name và group project topic. Chưa dùng props/state.

```
App
├── CourseHeader
└── StudentProfile
```

Bước B0 - AI-0 trước khi code

Không dùng AI trong 5 phút đầu. Viết ra câu trả lời:

Câu hỏi AI-0	Câu trả lời của bạn
Dữ liệu nào cần hiển thị?	_____
App sẽ render những component nào?	_____
File component nên đặt ở đâu?	_____
Component nào cần export/import?	_____
Bạn sẽ kiểm tra thành công bằng evidence nào?	_____

Bước B1 - Clean starter có kiểm soát

Sửa App.jsx về baseline tối giản. Không xóa project hàng loạt.

```
function App() {
  return (
    <h1>SBA301</h1>
  );
}

export default App;
```

CHECK

Save → browser cập nhật → không lỗi console. Nếu fail, sửa ngay trước khi đi tiếp.

Bước B2 - Tạo CourseHeader

src/components/CourseHeader.jsx

Gợi ý tối thiểu:

```
function CourseHeader() {
  const courseCode = "SBA301";
  const topic = "ReactJS Fundamentals";

  return (
    <header>
      <h1>{courseCode}</h1>
      <p>{topic}</p>
    </header>
  );
}

export default CourseHeader;
```

CHECK

Không cần đẹp. Mục tiêu là component + variable + JSX expression + export hợp lệ.

Bước B3 - Tạo StudentProfile

```
src/components/StudentProfile.jsx
```

Bạn tự triển khai. Tối thiểu có student name và project topic. Dùng ít nhất một object property để luyện JSX expression.

```
const student = {
  name: "Your Name",
  project: "Your Group Project"
};

// Render student.name và student.project bằng JSX expression
```

Bước B4 - Compose trong App.jsx

```
import CourseHeader from "../components/CourseHeader";
import StudentProfile from "../components/StudentProfile";

function App() {
  return (
    <>
      <CourseHeader />
      <StudentProfile />
    </>
  );
}

export default App;
```

CHECK

Browser phải hiển thị cả 2 component. Console không có relevant error.

Bước B5 - Predict before run

Trước khi save, dự đoán output của đoạn sau:

```
const a = 2;
const b = 3;

return <p>{a + b}</p>;
```

Dự đoán: _____ Kết quả thực tế: _____ Giải thích: _____

Bước B6 - Modify & observe

12. Thay giá trị Project Topic.
13. Save.
14. Quan sát browser update.
15. Ghi chú: đây vẫn là static source-code data, chưa phải React State.

Bước B7 - Browser evidence

Evidence	Kết quả
----------	---------

URL local	_____
CourseHeader hiển thị	PASS / FAIL
StudentProfile hiển thị	PASS / FAIL
JSX expression hoạt động	PASS / FAIL
Console relevant errors	0 / Có: _____
Screenshot/evidence path (nếu lớp yêu cầu)	_____

12. Cách phối hợp pair/group trong Session 02

Luân phiên vai trò

Mục tiêu không phải một bạn code cho cả nhóm. Sau mỗi checkpoint, đổi vai để mọi thành viên đều gõ lệnh, đọc code và giải thích.

Vai trò	Trách nhiệm	Đổi vai khi
Driver	Gõ command/code theo kế hoạch	Sau mỗi checkpoint A/B
Navigator	Đọc yêu cầu, dự đoán bước tiếp theo, nhắc không vượt scope	Sau 10-15 phút
Reviewer	Kiểm tra browser/console/evidence, hỏi “vì sao?”	Sau khi feature chạy
Explainer	Giải thích flow cho bạn khác mà không nhìn AI	Cuối hoạt động

Peer Review 2 phút

Một sinh viên giải thích main.jsx → App → CourseHeader/StudentProfile → JSX → browser. Người nghe chỉ được hỏi: “file nào?”, “ai render?”, “dữ liệu ở đâu?”, “evidence nào chứng minh?”.

13. Debugging - luôn tìm stage lỗi đầu tiên

```

React page fails
↓
Does npm run dev start?
↓
Can browser reach dev URL?
↓
Terminal compile error?
↓
Browser console error?
↓
Import path correct?
↓
Component exported?
↓
JSX valid?
↓
Expected data rendered?

```

13.1. Debugging loop bắt buộc

```

UI wrong
↓
Check terminal
↓
Check browser console
↓
Read FIRST meaningful error
↓
Identify file/line
↓
Make ONE change
↓
Verify again

```

Symptom	Likely cause	Smallest diagnostic / fix	Verification
node/npm not recognized	Environment/PATH	node -v; npm -v; không sửa code	Commands trả version
npm run dev fails outside project	Wrong directory	Get-Location; kiểm tra package.json	Dev server start
Cannot resolve module	Missing/wrong import path	Đọc path từ current file	Compile success
Welcome is not defined	Missing import	Kiểm tra import trong App.jsx	Component renders
<welcome /> behaves wrong	Lowercase component name	Đổi thành <Welcome />	User component render
Blank page + syntax error	Invalid JSX	Đọc first terminal/console error	No relevant error
Component shows nothing	Missing return	Kiểm tra function return JSX	Expected UI appears
Port in use	Port conflict	Đọc Vite terminal; dùng URL/port được cấp	Browser reaches server

14. Git checkpoint - chỉ commit khi trạng thái có ý nghĩa

Trước khi commit

App phải chạy và evidence cơ bản PASS. Commit không phải là cách “che” code lỗi.

```
git status
```

Nếu project chưa là Git repository và workflow lớp yêu cầu tạo local repo:

```
git init
```

```

git add .
git status
git commit -m "Build first React component structure"
git log -1

```

Git evidence	Ghi nhận
--------------	----------

Repository initialized	YES / NO / Already existed
Commit message	_____
Commit hash	_____
Files intentionally NOT committed	node_modules / khác: _____

Git ≠ GitHub upload

git commit tạo checkpoint trong repository local. Push/remote repository là bước khác và có thể để homework nếu lớp chưa cấu hình remote.

15. Sử dụng AI đúng cách trong Session 02

Course rule

AI = tutor/debugger/reviewer. AI không thay việc hiểu, test, evidence và giải thích của sinh viên.

15.1. AI-0 rule

- Tự viết/đoán trước.
- Chạy và lấy evidence.
- Chỉ sau đó hỏi AI.
- So sánh AI với evidence và course mental model.
- Tự sửa và giải thích lại không cần AI.

15.2. Prompt giải thích JSX

```
Act as a React tutor.
Explain this beginner React component:
[CODE]

For each line identify:
- JavaScript
- JSX
- React component
- import/export
- rendered data

Do not rewrite the component.
Then ask me 5 questions to test my understanding.
```

15.3. Prompt chẩn đoán lỗi

My React/Vite application failed.

Expected:
[EXPECTED]

Actual:
[ACTUAL]

Terminal:
[TERMINAL OUTPUT]

Browser Console:
[ERROR]

Relevant file:
[CODE]

Do not rewrite the project.
Identify the FIRST failing stage and one smallest diagnostic action.

15.4. Prompt review component - chống overengineering

Review this first React application.

Do not introduce props, state, hooks, context, router, TypeScript, Redux or other advanced tools.

Check only:

1. component naming
2. JSX correctness
3. imports/exports
4. unnecessary duplication
5. beginner readability

Explain issues before suggesting changes.

15.5. Hallucination check

Phát biểu cần đánh giá

“React stores JSX directly in the SQL Server database when a component renders.”

16. Đánh giá đúng/sai mà không AI.

17. Hỏi AI và ghi lại kết luận.

18. Đối chiếu kiến trúc Buổi 01.

19. Viết phát biểu đúng bằng lời của bạn.

Kết luận của tôi: _____

16. Các hiểu lầm cần loại bỏ

Hiểu lầm	Sửa lại
React là programming language	React là JavaScript library cho UI
JSX chính là HTML	JSX là syntax extension HTML-like trong JavaScript/React
Vite = React	Vite là tooling; React là UI library

npm install chạy app	npm install cài dependencies
npm run dev cài package	npm run dev chạy dev script/server
Node.js khiến project thành Node backend	Node chủ yếu phục vụ tooling trong frontend development ở buổi này
Mọi HTML tag đều là React component	lowercase thường là DOM element; capitalized dùng cho user component
React tự kết nối Spring Boot	Sau này phải chủ động dùng HTTP/API
Browser blank → xóa project tạo lại	Đọc terminal/console và tìm first failing stage trước
Git commit tự upload GitHub	Commit local; push là bước riêng

17. Completion checklist - “chạy được” chưa đủ

- ☐ npm run dev succeeds.
- ☐ Browser mở đúng local URL.
- ☐ App component render.
- ☐ CourseHeader render.
- ☐ StudentProfile render.
- ☐ Có ít nhất một JavaScript variable được render qua JSX expression.
- ☐ Có ít nhất một object property được render.
- ☐ Import/export hợp lệ.
- ☐ Không có relevant browser console error.
- ☐ Có Git commit có ý nghĩa.
- ☐ Tự giải thích được main.jsx → App → child component → JSX → browser trong ≤ 60 giây.

18. Human Verification Gate - không dùng AI

Câu hỏi	Tự đánh giá
1. React là gì?	PASS / REVIEW
2. Vite khác React thế nào?	PASS / REVIEW
3. npm create vite làm gì về mặt khái niệm?	PASS / REVIEW
4. npm install làm gì?	PASS / REVIEW
5. npm run dev làm gì?	PASS / REVIEW
6. package.json là gì?	PASS / REVIEW
7. main.jsx có vai trò gì?	PASS / REVIEW
8. App.jsx có vai trò gì?	PASS / REVIEW
9. JSX là gì?	PASS / REVIEW
10. {} trong JSX dùng để làm gì?	PASS / REVIEW
11. Function component là gì?	PASS / REVIEW

12. Vì sao tên component viết hoa?	PASS / REVIEW
13. import/export có vai trò gì?	PASS / REVIEW
14. Khi blank page, evidence đầu tiên cần lấy là gì?	PASS / REVIEW
15. Vì sao phải commit sau working milestone?	PASS / REVIEW

Gate rule

Nếu app chạy nhưng bạn không giải thích được các khái niệm trên, Session 02 chưa hoàn thành về mặt học tập.

19. Exit Ticket (5 câu)

20. React và Vite khác nhau như thế nào?
21. Điền flow: index.html → _____ → App → Component → Browser.
22. `{courseName}` trong JSX có ý nghĩa gì?
23. Tại sao `- 24. Giải thích vai trò của `npm create vite`, `npm install`, `npm run dev`.

20. Phiếu kết quả cá nhân Session 02

Mục	Kết quả
Node version	_____
npm version	_____
Vite project	PASS / FAIL
React dev URL	_____
Root/start file	_____
Main application component	_____
Components created	_____
JS variable rendered	_____
Object property rendered	_____
Relevant console errors	0 / _____
Git commit hash	_____
Một lỗi tôi đã tự debug	_____
Một điều AI gợi ý và tôi đã verify	_____

21. Reflection - viết 5 phút sau buổi học

- Before today, I thought React was ...
- Now I understand a Component as ...
- JSX is different from HTML because ...
- The role of Vite is ...
- One error I encountered was ...

- The evidence that helped me fix it was ...
- AI suggested ... and I verified it by ...
- My first React commit was ...

22. Home Practice trước Session 03

Task A - Thêm ProjectSummary component

- Hiển thị project name.
- Hiển thị target users.
- Hiển thị một core feature.
- Không dùng props/state.

Task B - JSX expressions

- Ít nhất 1 string variable.
- Ít nhất 1 numeric expression.
- Ít nhất 1 object property.

Task C - Git

- Chạy và verify trước.
- Commit với meaningful message.
- Ghi commit hash vào evidence.

Task D - Explain

60-second challenge

Không nhìn note/AI, giải thích: main.jsx → App → child component → JSX → browser.

23. Chuyển kết quả sang Group Project

Project Transfer

Mỗi nhóm phải có một placeholder React UI đại diện cho topic đã chọn ở Buổi 01. Chỉ UI tĩnh; chưa backend.

GROUP PROJECT TITLE
Target Users: ...
Core Feature: ...

Project deliverable cuối Session 02	PASS/FAIL
React app chạy	
Tên project xuất hiện	
Ít nhất 2 component	
Git commit tồn tại	
Mỗi thành viên giải thích được basic flow	

24. Timeline 135 phút dành cho sinh viên

Thời gian	Bạn làm gì	Output
00-10	Recall Buổi 01 + frontend mental model	6 câu tự trả lời
10-25	React, declarative UI, component thinking	Mental model
25-38	Vite + npm + project creation plan	Biết vai trò từng command
38-53	Tạo/chạy Vite project	Baseline URL PASS
53-68	Đọc project structure	main.jsx/App.jsx flow
68-83	JSX + expressions + fragment	JSX mini examples
83-98	Function component + import/export	Component đầu tiên
98-113	My SBA301 Profile Application	2 components render
113-121	Debug terminal + console	Error evidence/repair
121-127	Git checkpoint	Commit
127-132	AI review + verify	Correction có evidence
132-135	Exit ticket	5 câu trả lời

25. Glossary tối thiểu cần nhớ

Term	Định nghĩa ngắn	Ví dụ trong Session 02
React	JavaScript library cho UI	App, CourseHeader
UI	Phần người dùng thấy/tương tác	Profile page
Declarative UI	Mô tả UI theo dữ liệu	{courseName} → h1
Component	Khối UI có thể tổ chức/tái sử dụng	StudentProfile
Component Tree	Quan hệ component cha-con	App → CourseHeader
JSX	Cú pháp HTML-like trong JavaScript/React	{name}
Expression	Biểu thức tạo ra giá trị	{1+2}, {student.name}
Render	Biến component/JSX thành UI	React renders App
main.jsx	Entry của React rendering trong starter	render <App />
App.jsx	Top-level starter component	Compose child components
Import	Đưa exported value vào module hiện tại	import CourseHeader...
Export	Cho module khác dùng value/component	export default ...
Vite	Frontend development/build tooling	dev server
Development Server	Server phục vụ app khi develop	local Vite URL
package.json	Metadata/scripts/dependencies	npm scripts
Dependency	Package project cần	react, react-dom...
node_modules	Dependencies đã cài	không sửa tay

Git	Version control	git status/commit
Commit	Checkpoint có lịch sử	Build first React...

26. Cầu nối sang Session 03

Vấn đề mới

Sau hôm nay App có thể bắt đầu chứa nhiều JSX. Câu hỏi của buổi tối: làm thế nào chia nhỏ, tái sử dụng và tổ chức component tốt hơn?

SESSION 03
 React Components
 + Component Composition
 + Reusable UI
 + React-Bootstrap

27. Quick Answer Key - Exit Ticket

React là UI library; Vite là development/build tooling dùng để scaffold/run/build frontend project.

index.html → main.jsx/React root → App → Component → Browser.

`{courseName}` nhúng giá trị JavaScript expression vào JSX.

Tên bắt đầu chữ hoa được React hiểu theo hướng user-defined component; lowercase thường được hiểu là DOM element.

`npm create vite` : scaffold project; `npm install` : cài dependencies; `npm run dev` : chạy development server/script.

END OF SESSION 02 STUDENT GUIDE